DESENVOLVIMENTO MOBILE · MÓDULO 02 · TRILHA KOTLIN · NÍVEL INICIANTE

Kotlin Essencial

Tipos, null safety, controle de fluxo, funções, orientação a objetos, coleções e funções de escopo

Prof. Dr. Francisco dos Santos Viana

São Luís – Maranhão – Brasil · 2026

Objetivos do módulo

Ao final deste módulo você será capaz de

- Declarar variáveis e usar os tipos básicos e a inferência de tipos do Kotlin.
- Escrever código seguro contra `NullPointerException` com *null safety*.
- Usar `if` e `when` como expressões, além de *ranges* e laços.
- Criar funções com parâmetros padrão, funções de extensão e lambdas.
- Modelar dados com `data class`, `enum`, `sealed` e interfaces.
- Manipular coleções com operações funcionais e funções de escopo.

Onde praticar

`https://play.kotlinlang.org` roda Kotlin no navegador, sem instalar nada. Os *Kotlin Koans* trazem exercícios guiados.

Roteiro

Neste módulo

- Fundamentos: linguagens de programação
- Primeiros passos
- Controle de fluxo
- Null safety
- Funções e lambdas
- Orientação a objetos
- Coleções
- Funções de escopo e erros
- Fechamento

Fundamentos: linguagens de programação

Paradigmas de programação

Paradigma	Ideia central
Imperativo	sequência de comandos que alteram o estado (*faça isto, depois aquilo*)
Orientado a objetos	dados e comportamentos agrupados em objetos que trocam mensagens
Funcional	funções puras, valores imutáveis, composição de funções
Declarativo	descreve **o que** se quer, não **como** obter (SQL, interfaces em Compose e Flutter)

Paradigma

Estilo de organizar o raciocínio e o código. Linguagens modernas são **multiparadigma**: Kotlin e Dart misturam imperativo, objetos e funcional.

Dica

Use o paradigma que deixa cada parte mais clara: objetos para modelar o domínio, funções para transformar dados, declarativo para descrever a interface.

Um pouco de história

Fortran (1957) era imperativo; Lisp (1958), funcional; Simula (1967) e Smalltalk (anos 1970) criaram a orientação a objetos. Kotlin (2011, JetBrains) e Dart (2011, Google) herdaram ideias de todos.

Sistemas de tipos

Tipo

Conjunto de valores possíveis e das operações permitidas sobre eles. Somar dois `Int` faz sentido; somar um `Int` a uma `Bancada`, não.

Classificação	Significado
Estática	tipos verificados na compilação (Kotlin, Dart, Java)
Dinâmica	tipos verificados na execução (Python, JavaScript)
Forte	conversões perigosas são proibidas ou explícitas
Inferência	o compilador deduz o tipo (`val x = 3`)

Por que tipagem estática ajuda

Muitos erros aparecem **antes** de o app rodar; a IDE completa código e refatora com segurança; o tipo documenta a intenção.

Solidez (*soundness*)

Um sistema de tipos é sólido quando garante que, se o programa compila, uma variável nunca terá em execução um valor de outro tipo. O Dart 3 é sólido, inclusive quanto a nulos.

O “erro de um bilhão de dólares”: o nulo

Um pouco de história

Tony Hoare introduziu a referência nula na linguagem ALGOL W em 1965. Décadas depois, chamou-a de seu “erro de um bilhão de dólares”, pelo custo das falhas que ela causou.

Null safety

O sistema de tipos separa tipos que **podem** ser nulos (`String?`) dos que **não podem** (`String`). Usar um valor possivelmente nulo sem tratar vira erro de compilação, não falha em produção.




```
if (x != null)
x ?: padrão
```

Análise de fluxo

Depois de um `if (x != null)`, o compilador sabe que `x` não é nulo naquele trecho e permite usá-lo diretamente (*smart cast* no Kotlin, promoção de tipo no Dart).

Atenção

O operador `!!` (Kotlin) ou `!` (Dart) desliga a proteção: só use quando tiver certeza.

Os pilares da orientação a objetos

Pilar	Significado e exemplo
Encapsulamento	esconder o estado interno e expor operações: a `Reserva` só muda pelo método `cancelar()`
Abstração	mostrar o essencial e ocultar detalhes: `FonteLeituras` sem dizer se é BLE ou MQTT
Herança	uma classe especializa outra: `SensorTemperatura` herda de `Sensor`
Polimorfismo	o mesmo comando, comportamentos diferentes: `fonte.leituras()` em qualquer implementação

Objeto e classe

A **classe** é o molde (atributos e métodos); o **objeto** é uma instância concreta, com estado próprio.

Composição em vez de herança

Prefira montar objetos com outros objetos ("tem um") a criar hierarquias profundas ("é um"): mudanças ficam localizadas e o código, mais flexível.

Analogia

Uma tomada é uma abstração: qualquer aparelho com o plugue certo funciona (polimorfismo), e ninguém precisa conhecer a fiação dentro da parede (encapsulamento).

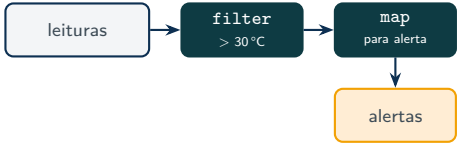
Programação funcional

Função pura

Para a mesma entrada, devolve sempre a mesma saída e não causa efeitos colaterais (não altera variáveis externas, não grava em disco, não acessa a rede).

Funções de primeira classe e de ordem superior

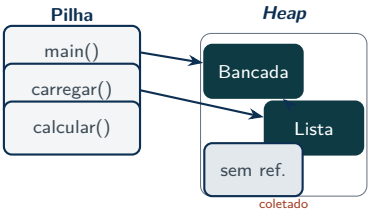
Funções são valores: podem ser guardadas em variáveis, passadas como argumento e devolvidas. Uma função que recebe ou devolve outra é de **ordem superior** (`map`, `filter`).



Por que importa no mobile

Funções puras são fáceis de testar e seguras entre *threads*; transformar listas com `map`/`filter` é mais legível que laços; interfaces declarativas tratam a tela como uma função do estado (Módulo 05).

Memória: pilha, *heap* e coleta de lixo



Pilha e *heap*

A **pilha** guarda as chamadas de função e suas variáveis locais, liberadas ao retornar. O ***heap*** guarda os objetos, que vivem enquanto houver referências a eles.

Coleta de lixo

O *runtime* encontra objetos que não são mais alcançáveis a partir das referências vivas e libera sua memória automaticamente.

Gerações

A maioria dos objetos morre jovem. Por isso o ART e a VM do Dart separam objetos novos e antigos e coletam os novos com frequência e rapidez.

Imutabilidade e igualdade

Imutável

Objeto que não muda depois de criado. Para "alterar", cria-se uma cópia com o campo novo (`copy` no Kotlin, `copyWith` no Dart).

Vantagens

- Ninguém altera o objeto "pelas costas".
- Seguro para compartilhar entre *threads* e *isolates*.
- Comparar o antes e o depois de um estado fica trivial (Módulo 07).

Igualdade de valor e de identidade

Identidade: as duas variáveis apontam para o **mesmo** objeto. **Valor**: objetos diferentes com o mesmo conteúdo. `data class` (Kotlin) e *records* (Dart) comparam por valor.

Lista mutável compartilhada

Dois componentes recebem a mesma lista; um a altera e o outro mostra dados que não esperava. Exponha listas somente leitura.

Primeiros passos

Por que Kotlin?

- Criada pela JetBrains; linguagem preferencial do Android desde 2019.
- **Concisa**: menos código repetitivo que Java.
- **Segura**: o sistema de tipos distingue o que pode e o que não pode ser nulo.
- **Interoperável**: usa qualquer biblioteca Java.
- **Multiplataforma**: JVM, Android, iOS, web e desktop (Módulo 19).
- Suporte nativo a programação assíncrona com *coroutines* (Módulo 06).

Java

```
public class Sensor {
  private final String nome;
  public Sensor(String nome) {
    this.nome = nome;
  }
  public String getNome() {
    return nome;
  }
  // + equals, hashCode, toString...
}
```

Kotlin

```
data class Sensor(val nome: String)
```

Variáveis e tipos básicos

Variaveis.kt

```kotlin
fun main() {
  val bancada = 3          // imutável (prefira!)
  var temperatura = 24.5   // mutável
  temperatura += 0.3

  val nome: String = "Osciloscópio"  // tipo explícito
  val ligado = true                  // Boolean
  val leituras: Long = 1_000_000L    // separador _
  val letra: Char = 'A'

  // String templates
  println("Bancada $bancada: $temperatura °C")
  println("Nome tem ${nome.length} letras")

  // Texto multilinha
  val relatorio = """
      |Equipamento: $nome
      |Estado: ${if (ligado) "ON" else "OFF"}
  """.trimMargin()
}
```

val *versus* var

`val` cria uma referência que não pode ser reatribuída. `var` pode. Use `val` por padrão e só troque se precisar.

Tipo	Exemplo
`Int` / `Long`	`42` / `42L`
`Double` / `Float`	`3.14` / `3.14f`
`Boolean`	`true` / `false`
`Char` / `String`	`'a'` / `"texto"`

Dica

Conversões são explícitas: `x.toInt()`, `"12".toDouble()`, `"abc".toIntOrNull()`.

Controle de fluxo

`if` e `when` são expressões

`if` como expressão

```
val status = if (temp > 80.0) "ALERTA" else "OK"

val nivel = if (temp < 20) {
  "frio"
} else if (temp < 30) {
  "normal"
} else {
  "quente"
}
```

Dica

Kotlin não tem operador ternário (`a ? b : c`): o `if` já devolve valor.

`when`: o switch turbinado

```
fun classificar(t: Double) = when {
  t < 0       -> "abaixo da faixa"
  t in 0.0..35.0 -> "normal"
  else        -> "acima da faixa"
}

fun descrever(x: Any): String = when (x) {
  is Int    -> "inteiro $x"
  is String -> "texto de ${x.length} letras"
  1, 2, 3   -> "pequeno"
  else      -> "desconhecido"
}
```

Ranges e laços

Laços

```
for (i in 1..5) print(i)          // 1 2 3 4 5
for (i in 0 until 5) print(i)     // 0 1 2 3 4
for (i in 10 downTo 0 step 2) print(i)
for (i in 0..<5) print(i)         // igual a until

val sensores = listOf("DHT22", "BMP280", "LDR")
for (s in sensores) println(s)
for ((i, s) in sensores.withIndex()) println("$i: $s")

var tentativas = 0
while (tentativas < 3) {
  tentativas++
}
repeat(3) { println("ping $it") }
```

Ranges

`a..b` inclui `b`; `a until b` e `a..<b` excluem `b`. O operador `in` testa se um valor pertence ao intervalo:

```
if (tensao in 4.75..5.25) { ...}
```

Dica

Em código Android você raramente escreverá laços: operações como `map` e `filter` (seção 6) costumam ser mais claras.

Null safety

Tipos anuláveis

Null safety

```
var nome: String = "LDR"
// nome = null          // ERRO de compilação

var apelido: String? = null   // pode ser nulo
// apelido.length       // ERRO: pode ser nulo
// 1) chamada segura: devolve null se for null
val tam: Int? = apelido?.length
// 2) elvis: valor padrão quando null
val tam2: Int = apelido?.length ?: 0

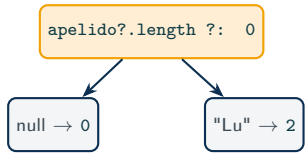
// 3) smart cast após checagem
if (apelido != null) {
  println(apelido.length)    // aqui é String
}

// 4) let: executa só se não for null
apelido?.let { println("Apelido: $it") }

// 5) !! afirma que não é null (evite!)
val tam3 = apelido!!.length  // lança NPE se null
```

A ideia

O compilador sabe se um valor pode ser nulo. `String` nunca é nulo; `String?` pode ser. Você é **obrigado** a tratar o caso nulo antes de usar.



Erro comum

Usar `!!` para "calar" o compilador traz de volta o `NullPointerException`.

Funçõesе lambdas

Funções

Funcoes.kt

```kotlin
// Forma completa
fun media(valores: List<Double>): Double {
  return valores.sum() / valores.size
}

// Corpo de expressão única
fun celsiusParaF(c: Double) = c * 9 / 5 + 32

// Parâmetros com valor padrão
fun conectar(
  host: String,
  porta: Int = 1883,
  tls: Boolean = false
) { /* ... */ }

conectar("broker.local")
conectar("broker.local", tls = true)  // nomeado

// Função sem retorno útil devolve Unit
fun log(msg: String) { println(msg) }
```

Funções de extensão

```kotlin
// "adiciona" um método a uma classe existente
fun Double.emCelsius() = "%.1f °C".format(this)

val MAC = Regex("([0-9A-F]{2}:){5}[0-9A-F]{2}")
fun String.ehMacValido() = matches(MAC)

println(24.567.emCelsius())   // 24.6 °C
println("24:6F:28:AB:CD:EF".ehMacValido())
```

Extensões

Não modificam a classe original: são funções comuns com uma sintaxe mais legível. São muito usadas no Android (`Modifier.padding`, `Context.getString`...).

Lambdas e funções de ordem superior

Lambdas

```
// Lambda: função anônima atribuída a uma variável
val dobro: (Int) -> Int = { x -> x * 2 }
val soma = { a: Int, b: Int -> a + b }

// Com um único parâmetro, use "it"
val ehPar: (Int) -> Boolean = { it % 2 == 0 }

// Função de ordem superior: recebe uma função
fun medir(n: Int, acao: () -> Unit): Long {
  val inicio = System.nanoTime()
  repeat(n) { acao() }
  return System.nanoTime() - inicio
}

// "Trailing lambda": a última lambda sai dos ()
val tempo = medir(1000) {
  calcularFFT()
}
```

Tipos de função

`(Int) -> Int` lê-se "recebe um `Int` e devolve um `Int`". `() -> Unit` não recebe nada e não devolve nada.

Isto é Compose!

Toda a interface do Compose usa *trailing lambdas*:

```
Button(onClick = { ligar() }) {
  Text("Ligar")
}
```

`onClick` recebe uma lambda e o conteúdo do botão é outra lambda.

Orientação a objetos

Classes, construtores e propriedades

Classes

```
class Equipamento(
  val patrimonio: String,        // propriedade imutável
  var descricao: String,         // propriedade mutável
  private var horasDeUso: Int = 0
) {
  val precisaManutencao: Boolean   // calculada
    get() = horasDeUso > 500

  init {
    require(patrimonio.isNotBlank()) { "Patrimônio
      obrigatório" }
  }

  fun registrarUso(horas: Int) { horasDeUso += horas }
}

val osc = Equipamento("IFMA-0042", "Osciloscópio")
osc.registrarUso(12)
println(osc.precisaManutencao)
```

- O **construtor primário** fica no cabeçalho da classe.
- `val`/`var` no construtor já criam as propriedades.
- `init` roda na criação do objeto.
- Não existe `new`.
- Visibilidade: `public` (padrão), `private`, `protected`, `internal` (mesmo módulo).

Dica

`require` valida argumentos (lança `IllegalArgumentException`); `check` valida estado (lança `IllegalStateException`).

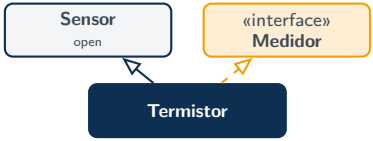
Herança e interfaces

Herança

```
interface Medidor {
  val unidade: String
  fun ler(): Double
  fun formatado() = "${ler()} $unidade"  // padrão
}

// classes são final por padrão: use open
open class Sensor(val pino: Int) {
  open fun calibrar() { println("calibrando $pino") }
}

class Termistor(pino: Int) : Sensor(pino), Medidor {
  override val unidade = "°C"
  override fun ler() = 25.0
  override fun calibrar() {
    super.calibrar()
    println("ajustando curva beta")
  }
}
```



Regras

- Classes e métodos são **final** por padrão; use **open** para permitir herança.
- Herança simples de classe; múltiplas interfaces.
- Interfaces podem ter implementação padrão.

data class, object e companion object

Data class

```
data class Leitura(
  val sensorId: String,
  val valor: Double,
  val instante: Long = System.currentTimeMillis()
)

val a = Leitura("t1", 24.5, 1000)
val b = Leitura("t1", 24.5, 1000)
println(a == b)             // true: compara conteúdo
println(a)                  // Leitura(sensorId=t1, ...)
val c = a.copy(valor = 25.1)  // cópia alterada
val (id, valor) = a         // desestruturação
```

data class

Gera `equals`, `hashCode`, `toString`, `copy` e `componentN`. É a base dos modelos e estados de tela.

object e companion

```
// Singleton: uma única instância
object Config {
  const val BROKER = "mqtt.ifma.local"
  var debug = false
}

class Leitor private constructor(val id: String) {
  companion object {   // "estático"
    fun criar(id: String) = Leitor(id.trim())
  }
}

val l = Leitor.criar(" t1 ")
println(Config.BROKER)
```

`enum class` e `sealed` para modelar estados

Enum

```
enum class StatusBancada(val cor: String) {
  LIVRE("verde"),
  RESERVADA("âmbar"),
  MANUTENCAO("vermelho");

  fun disponivel() = this == LIVRE
}

val s = StatusBancada.valueOf("LIVRE")
StatusBancada.entries.forEach { println(it) }
```

Dica

Use `enum` quando os casos são fixos e **não carregam dados diferentes** entre si.

Sealed interface

```
sealed interface Resultado {
  data object Carregando : Resultado
  data class Sucesso(val dados: List<Leitura>)
    : Resultado
  data class Erro(val msg: String) : Resultado
}

// when exaustivo: sem else
fun render(r: Resultado) = when (r) {
  Resultado.Carregando -> "Carregando..."
  is Resultado.Sucesso -> "${r.dados.size} itens"
  is Resultado.Erro    -> "Falha: ${r.msg}"
}
```

Por que sealed?

Cada caso carrega dados próprios e o `when` exaustivo obriga a tratar todos: um caso novo vira erro de compilação onde faltar tratamento.

Coleções

Listas, conjuntos e mapas

Coleções

```kotlin
// Somente leitura (preferível)
val nomes = listOf("DHT22", "BMP280", "LDR")
val ids = setOf(1, 2, 2, 3)          // {1, 2, 3}
val pinos = mapOf("led" to 2, "rele" to 4)

// Mutáveis
val fila = mutableListOf<Leitura>()
fila.add(Leitura("t1", 24.0))
fila += Leitura("t1", 24.2)

val cache = mutableMapOf<String, Double>()
cache["t1"] = 24.2
val v = cache["t2"] ?: 0.0     // get pode ser null

println(nomes[0])              // DHT22
println(nomes.getOrNull(9))    // null (sem exceção)
println(pinos.getValue("led")) // 2
```

Leitura *versus* mutável

`List` não tem `add`. `MutableList` tem. Exponha sempre a versão de leitura e mantenha a mutável privada.

Tipo	Característica
`List`	ordenada, aceita repetidos
`Set`	sem repetidos
`Map`	pares chave → valor

Operações funcionais

Pipeline de dados

```
val leituras = listOf(
  Leitura("t1", 24.1), Leitura("t2", 81.3),
  Leitura("t1", 24.9), Leitura("t3", 19.0),
  Leitura("t2", 79.8)
)

val alertas = leituras
  .filter { it.valor > 75 }              // seleciona
  .map { "${it.sensorId}: ${it.valor}" }  // transforma

val mediaPorSensor = leituras
  .groupBy { it.sensorId }
  .mapValues { (_, l) -> l.map { it.valor }.average() }

val maior = leituras.maxByOrNull { it.valor }
val total = leituras.sumOf { it.valor }
val desc = leituras.sortedByDescending { it.valor }
val temAlerta = leituras.any { it.valor > 80 }
val t3 = leituras.firstOrNull { it.sensorId == "t3" }
```

Função	Resultado
`map`	transforma cada item
`filter`	mantém os que passam
`groupBy`	agrupa em um `Map`
`sortedBy`	nova lista ordenada
`sumOf`, `average`	agregações
`any`, `all`, `none`	testes lógicos
`firstOrNull`	primeiro que satisfaz
`associateBy`	`Map` por chave
`chunked`, `windowed`	blocos e janelas

Dica

`windowed(5)` calcula médias móveis de leituras de sensor em uma linha.

Funções de escopo e erros

Funções de escopo

Função	Objeto	Retorna	Uso típico
`let`	`it`	lambda	tratar anuláveis
`apply`	`this`	objeto	configurar objeto
`also`	`it`	objeto	efeito extra (log)
`run`	`this`	lambda	calcular com o objeto
`with`	`this`	lambda	agrupar chamadas

Exemplos

```
val cfg = ConfigMqtt().apply {
  host = "broker.local"
  porta = 1883
}

val leitura = ler().also {
  Log.d(TAG, "lida: $it")
}

dispositivo?.let { conectar(it) }

val resumo = with(leitura) {
  "$sensorId = $valor"
}
```

Dica

Use com moderação: muitas funções de escopo aninhadas prejudicam a leitura.

Tratamento de erros

try é expressão

```
val valor: Double = try {
  entrada.toDouble()
} catch (e: NumberFormatException) {
  Log.w(TAG, "valor inválido: $entrada")
  0.0
} finally {
  fecharArquivo()
}

fun lerSensor(): Double {
  if (!conectado) throw
      IllegalStateException("desconectado")
  return 24.5
}
```

Result e runCatching

```
val r: Result<Double> = runCatching { lerSensor() }

r.onSuccess { println("ok: $it") }
 .onFailure { println("falhou: ${it.message}") }

val v = r.getOrDefault(Double.NaN)
val txt = r.fold(
  onSuccess = { "%.1f".format(it) },
  onFailure = { "--" }
)
```

Conceito

Kotlin não tem *checked exceptions*: o compilador não obriga a capturar nada. Por isso é importante documentar e tratar falhas de rede, arquivo e conversão.

Fechamento

Resumo

Resumo

- Prefira `val`; a inferência de tipos deixa o código enxuto sem perder segurança.
- `T?` marca valores anuláveis; use `?.`, `?:`, `let` e *smart casts*, e evite `!!`.
- `if`, `when` e `try` são expressões que devolvem valor.
- Parâmetros padrão e nomeados, extensões e lambdas são a base da API do Compose.
- `data class` para dados, `sealed` para estados, `object` para singletons.
- Coleções somente leitura + `map`/`filter`/`groupBy` substituem a maioria dos laços.

Glossário

Termo	Significado
Paradigma	estilo de organizar o código (imperativo, objetos, funcional, declarativo)
Tipagem estática	tipos verificados na compilação
Inferência de tipos	o compilador deduz o tipo a partir do valor
Null safety	distinção entre tipos anuláveis (`String?`) e não anuláveis
Smart cast	conversão automática após uma verificação de tipo ou de nulo
`data class`	classe de dados com `equals`, `hashCode`, `copy` e `toString` gerados
`sealed class`	hierarquia fechada: o `when` sabe todos os casos
Função de ordem superior	função que recebe ou devolve outra função
Coleta de lixo	liberação automática de objetos sem referências

Questões de revisão

1. Que vantagens a tipagem estática traz para um app que vai crescer?
2. Por que o nulo foi chamado de "erro de um bilhão de dólares"? Como o Kotlin o trata?
3. Qual a diferença entre `val` e `var`, e por que preferir `val`?
4. Dê um exemplo de encapsulamento e um de polimorfismo no LabMonitor.
5. O que é uma função pura? Por que ela é fácil de testar?
6. Para que serve uma `sealed class` ao modelar o estado de uma tela?
7. Explique a diferença entre igualdade de valor e de identidade com `data class`.
8. O que acontece com um objeto quando nenhuma variável aponta mais para ele?

Exercícios

Exercício 1 — Conversões

Crie funções de extensão `Double.cParaF()`, `Double.fParaC()` e `Int.adcParaVolts(vref: Double = 3.3, bits: Int = 12)`, que converte uma leitura de ADC em tensão.

Exercício 2 — Coleções

Dada uma lista de `Leitura`, calcule: a média por sensor; o sensor com maior variância; a média móvel de 5 amostras do sensor `"t1"`; as leituras fora da faixa 10–40 °C.

Exercício 3 — Modelagem

Modele com `sealed interface` os estados de uma conexão BLE: *Desconectado*, *Procurando*, *Conectando(endereco)*, *Conectado(endereco, rssi)* e *Falha(motivo)*. Escreva uma função que gere a mensagem de cada estado com um `when` exaustivo.

LabMonitor — Etapa 2

Crie o pacote `model` com `Equipamento`, `Bancada`, `Leitura` e `enum StatusBancada`, mais uma lista de dados fictícios em `object DadosExemplo`.

Para aprofundar

- Documentação oficial: `https://kotlinlang.org/docs/home.html`
- Kotlin Koans: `https://play.kotlinlang.org/koans`
- Kotlin para desenvolvedores Android: `https://developer.android.com/kotlin`
- Convenções de código: `https://kotlinlang.org/docs/coding-conventions.html`
- Livro gratuito: *Kotlin Bootcamp for Programmers* (codelab do Google)

Respostas das questões de revisão

1. Erros de tipo aparecem na compilação, a IDE completa e refatora com segurança e os tipos documentam a intenção do código.
2. Porque referências nulas causaram incontáveis falhas e custos. O Kotlin separa `String` de `String?` e exige tratar o nulo (`?.`, `?:`, *smart cast*) antes do uso.
3. `val` não pode ser reatribuída; `var` pode. `val` evita mudanças inesperadas e facilita o raciocínio sobre o código.
4. Encapsulamento: `Reserva` só muda por `cancelar()`. Polimorfismo: `FonteBle` e `FonteFake` respondem a `leituras()` cada uma do seu jeito.
5. Devolve sempre o mesmo resultado para a mesma entrada, sem efeitos colaterais; o teste só precisa conferir entrada e saída.
6. Lista todos os estados possíveis (carregando, sucesso, erro); o `when` exaustivo obriga a tratar cada um.
7. Duas instâncias de `Bancada("B3")` são iguais por valor (`==` verdadeiro), mas não são o mesmo objeto (`===` falso).
8. Ele se torna inalcançável e é liberado pelo coletor de lixo em algum momento.

Fim do Módulo 02

Próximo: Módulo 03 (comum) — Engenharia de software e Git: requisitos, métodos ágeis, versionamento e colaboração.